

Trabajo Final Integrador



2 0 2 6

Propuesta de Proyecto y Repositorio

Grupo 152:

Tiziano Caamaño, Fabián Cardozo, Lautaro Cejas

Tutor:

Sergio Andres Antonini



3 0 / 0 8



Índice

Identificación del Problema y Propuesta de Solución.....	3
Contexto y Actores.....	3
Problema Central.....	3
Valor Agregado.....	4
Definición del Stack Tecnológico.....	5
Frontend.....	5
Backend.....	5
Base de datos.....	5
Alcance del Proyecto (Requisitos).....	6
Requisitos Funcionales (MVP).....	6
Requisitos No Funcionales.....	9
Fuera del Alcance (Out of Scope).....	10
Análisis de Viabilidad.....	11
Viabilidad Técnica.....	11
Viabilidad Operativa.....	11
Viabilidad Temporal.....	11
Uso Crítico de Herramientas de IA.....	12
Anexo.....	13



Identificación del Problema y Propuesta de Solución

El micro-comercio de proximidad representa un motor económico y social fundamental en múltiples ciudades y localidades de Argentina y el mundo. Sin embargo, mientras el comercio electrónico corporativo ha evolucionado hacia la hiper-automatización, el ecosistema barrial ha quedado marginado de esta transformación digital. Las plataformas de e-commerce tradicionales imponen barreras logísticas, requisitos fiscales inmediatos y curvas de aprendizaje que resultan incompatibles con la realidad del pequeño productor, el artesano o el trabajador de oficios. Ante esta brecha, la digitalización del sector se ha dado de forma accidentada, forzando el uso de redes sociales y aplicaciones de mensajería personal para sostener operaciones comerciales. Este proyecto nace con el objetivo de intervenir este escenario, proponiendo una solución de software diseñada específicamente para estructurar el comercio informal, respetando su escala y mitigando sus limitaciones tecnológicas.

Contexto y Actores

El comercio informal y de proximidad en entornos urbanos se gestiona actualmente de manera improvisada y fragmentada. Los actores principales involucrados en este ecosistema son, por un lado, los micro-comerciantes (productores artesanales, gastronómicos, oficios), que en una gran proporción incluyen a adultos mayores o personas con conocimientos tecnológicos limitados. Por otro lado, se encuentran los vecinos o compradores locales que buscan adquirir productos en su área inmediata. Al carecer de una infraestructura dedicada, ambos actores se ven obligados a interactuar y tranzar casi exclusivamente a través de estados efímeros de WhatsApp, redes sociales o grupos comunitarios.

Problema Central

La problemática central radica en la fricción transaccional y la desorganización operativa. La ausencia de un catálogo unificado y de un control de inventario en tiempo real genera que las ventas se diluyan en consultas manuales repetitivas ("¿sigue disponible?", "¿qué precio tiene?"). Esta dinámica satura al vendedor, exigiéndole una alta carga de tiempo, mientras que el comprador enfrenta la falta de garantías transaccionales y constantes malentendidos al coordinar la logística de entrega. Las plataformas de e-commerce tradicionales no representan una solución válida, ya que imponen barreras de entrada muy altas, logísticas de envío complejas y una formalidad fiscal que excluye directamente al pequeño productor barrial.



Valor Agregado

MeVe no busca ser una simple digitalización de un catálogo, sino que introduce un ecosistema O2O *Commerce* (Online-to-Offline) que inyecta ingeniería de software para resolver ineficiencias críticas. El valor agregado del proyecto se estructura en tres pilares funcionales:

- **Logística Inteligente (Gestor de Retiros):** Se erradica la incertidumbre de la coordinación por chat mediante un sistema de Click & Collect con cupos limitados. El comprador está obligado a reservar una franja horaria específica para retirar su pedido, lo que permite al vendedor ordenar su flujo de atención y evitar superposiciones.
- **Comercio Georreferenciado y Seguro:** La plataforma integra un motor de filtrado por proximidad para priorizar la oferta hiperlocal. Esto se complementa con la integración de MercadoPago (entorno Sandbox), garantizando el bloqueo temporal del stock y asegurando la transacción financiera antes de que ocurra el encuentro físico.
- **Accesibilidad Extrema:** Conscientes de la brecha digital de los actores, la interfaz cuenta con un diseño inclusivo desde su concepción. Incluye controles nativos para aplicar modo oscuro, ajuste de tamaño de fuente (zoom) sin romper la maqueta y paletas de alto contraste adaptadas para daltonismo, garantizando autonomía operativa para usuarios no técnicos.



Definición del Stack Tecnológico

La arquitectura de MeVe ha sido diseñada bajo un enfoque moderno, escalable y orientado a resolver la brecha digital del comercio informal. Para sostener el modelo de O2O Commerce (Online-to-Offline), se define el siguiente ecosistema tecnológico:

Frontend

- **Tecnologías base:** Se utilizará React como librería principal para la construcción de interfaces de usuario (UI) dinámicas y de una sola página (SPA). Todo el desarrollo del cliente estará tipado estáticamente mediante TypeScript, reduciendo errores en tiempo de ejecución y mejorando la mantenibilidad del código.
- **Accesibilidad y UX Inclusiva:** Se integrarán frameworks de diseño y librerías de componentes que permitan una parametrización profunda de la interfaz. Esto incluye la implementación nativa de **Modo Oscuro**, paletas de alto contraste aptas para diferentes tipos de **daltonismo** y soporte estructural para **zoom tipográfico** sin romper la maqueta. El objetivo tecnológico es lograr una accesibilidad extrema, garantizando que usuarios no técnicos puedan autogestionar su catálogo sin barreras cognitivas ni visuales.

Backend

- **Tecnologías base:** El núcleo de la lógica de negocio se desarrollará en Java utilizando el framework **Spring Boot**. Esta elección proporciona un entorno empresarial robusto, ideal para gestionar la alta concurrencia requerida por un marketplace.
- **Arquitectura y Seguridad:** Se implementará un patrón de arquitectura **MVC (Modelo-Vista-Controlador)** para la exposición de endpoints. La seguridad y el aislamiento de datos (vital para la arquitectura Multitenant del proyecto) estarán gestionados mediante Spring Security y el uso de **JWT (JSON Web Tokens)**, logrando una autenticación stateless (sin estado) que favorece la escalabilidad del servidor.

Base de datos

- **Tecnología:** Se empleará MongoDB Atlas.
- **Justificación:** Al tratarse de un marketplace hiperlocal que abarca desde viandas gastronómicas hasta manufacturas artesanales, los esquemas relacionales tradicionales resultan un obstáculo. La naturaleza NoSQL de MongoDB permite almacenar catálogos de productos completamente heterogéneos mediante documentos flexibles, adaptándose constantemente a la oferta de cada micro-comerciante.



Alcance del Proyecto (Requisitos)

El presente apartado delimita las capacidades que integran el Producto Mínimo Viable de MeVe y establece, con igual grado de detalle, aquello que queda deliberadamente fuera de esta iteración. Definir ambos límites con el mismo rigor resulta indispensable: en el desarrollo de plataformas de comercio, la incorporación progresiva de funcionalidades periféricas constituye la principal amenaza para la viabilidad de la entrega, y su delimitación temprana es la única forma efectiva de contenerla.

La premisa arquitectónica que ordena todo el alcance es que cada micro-comercio opere sobre su propio espacio de datos sin que ello fragmente la experiencia del comprador. Cada vendedor constituye, en consecuencia, un *tenant* con gestión aislada de su catálogo, su inventario y sus pedidos, mientras que la vitrina pública consolida la oferta de la totalidad de los comercios activos. El aislamiento multitenant opera sobre los módulos de gestión; la exploración del catálogo es, por definición, transversal.

Los requisitos se identifican mediante nomenclatura correlativa (RF para los funcionales, RNF para los no funcionales), a efectos de posibilitar su trazabilidad hacia los módulos de implementación y hacia el plan de pruebas del proyecto.

Requisitos Funcionales (MVP)

Los requisitos funcionales describen las capacidades de negocio que la plataforma debe proveer y se agrupan según los pilares definidos para el proyecto.

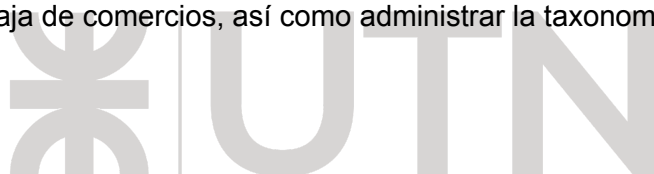
1.1. Arquitectura multitenant y aislamiento de datos

- **RF-01 · Constitución del tenant:** el sistema debe constituir a cada micro-comercio como una unidad independiente dentro de un entorno de infraestructura compartido.
- **RF-02 · Aislamiento en el acceso:** toda operación de gestión debe restringirse automáticamente al comercio asociado al usuario autenticado. La verificación no puede quedar delegada a la interfaz, sino que debe resolverse en la capa de servicios.
- **RF-03 · Vitrina consolidada:** el catálogo público debe agregar la oferta de todos los comercios activos y permanecer accesible sin autenticación previa.

1.2. Gestión de accesos y roles

El sistema opera bajo una asimetría deliberada: el vendedor se autentica porque administra información sensible y persistente, mientras que el comprador no lo hace. Exigirle registro reintroduciría la misma barrera de entrada que el proyecto se propone remover.

- **RF-04 Autenticación del vendedor:** el sistema debe permitir el registro y la autenticación de vendedores, emitiendo un token de sesión al validarse las credenciales.
- **RF-05 Perfiles diferenciados:** deben administrarse tres perfiles: SuperAdmin, Vendedor y Guest con permisos específicos verificados en cada punto de acceso.
- **RF-06 Operación sin registro:** el comprador debe poder completar una compra bajo el perfil Guest, aportando únicamente los datos mínimos necesarios para identificar el retiro.
- **RF-07 Administración de la plataforma:** el SuperAdmin debe poder gestionar el alta, la suspensión y la baja de comercios, así como administrar la taxonomía de categorías.



1.3. Gestión de catálogo e inventario

El catálogo constituye el activo principal del vendedor dentro de la plataforma, por lo que su administración debe resultar íntegramente autónoma y no depender de intervención técnica alguna.

- **RF-08 Administración de productos:** el vendedor debe poder crear, consultar, modificar y dar de baja productos, consignando denominación, descripción, precio, imagen, categoría y cantidad disponible.
- **RF-09 Baja lógica:** la eliminación de un producto debe ser lógica y no física, de modo que los pedidos históricos que lo contengan conserven su integridad.
- **RF-10 Publicación y despublicación:** el vendedor debe poder retirar temporalmente un producto de la vitrina sin eliminar su registro. La interrupción periódica de la oferta constituye una práctica habitual en el comercio de proximidad y el sistema debe acompañarla.
- **RF-11 Actualización de inventario:** el vendedor debe poder ajustar manualmente la cantidad disponible de cada producto desde su panel de gestión.
- **RF-12 Perfil del comercio:** el vendedor debe poder configurar la denominación, la descripción, la ubicación georreferenciada y los días y horarios de atención de su comercio.

1.4. Exploración hiperlocal y flujo de compra distribuido

El motor de filtrado por proximidad no constituye un accesorio de la interfaz sino el criterio principal de ordenamiento de la oferta. Es este comportamiento el que distingue a MeVe de los tableros de publicaciones generalistas, en los que la cercanía resulta apenas un atributo declarado por el que publica.

- **RF-13 Exploración del catálogo:** el comprador debe poder recorrer el catálogo unificado y consultar el detalle de cada producto y de su comercio de origen.
- **RF-14 Búsqueda textual:** el sistema debe proveer búsqueda sobre la denominación y la descripción de los productos publicados.
- **RF-15 Ordenamiento por proximidad:** los resultados deben ordenarse en función de la distancia entre la ubicación del comprador y la del comercio, resuelta mediante consulta geoespacial sobre índice dedicado.
- **RF-16 Filtrado:** el comprador debe poder acotar los resultados por categoría, por disponibilidad y por radio máximo de distancia.
- **RF-17 Carrito distribuido:** el carrito debe agrupar los ítems por comercio. Ante la selección de productos pertenecientes a comercios distintos, el sistema debe generar un pedido independiente por cada uno, preservando la autonomía operativa de cada vendedor.
- **RF-18 Registro del pedido:** cada pedido debe registrarse con su detalle de ítems, cantidades, importe total, comercio asociado, datos de contacto del comprador y franja horaria de retiro.

1.5. Logística inteligente (gestor de retiros)

Este módulo concentra el aporte central del proyecto. Sustituye la coordinación conversacional (origen de la mayor parte de la fricción descrita en el planteo del problema) por un mecanismo de asignación de capacidad que el vendedor configura una sola vez y el sistema administra de manera automática.

- **RF-19 Configuración de la atención:** el vendedor debe poder definir sus franjas horarias y la capacidad máxima de atención admitida por cada una de ellas.
- **RF-20 Oferta de franjas disponibles:** el sistema debe presentar al comprador únicamente aquellas franjas que conserven cupo, bloqueando de manera automática los turnos excedentes.
- **RF-21 Bloqueo preventivo:** la reserva de una franja debe comprometer temporalmente el stock involucrado y descontar el cupo correspondiente, situando el pedido en estado PENDIENTE_PAGO.
- **RF-22 Liberación automática:** una tarea programada debe detectar los pedidos cuya ventana de confirmación (fijada en quince minutos) haya caducado y restituir el stock y el cupo a la

disponibilidad general. Sin este mecanismo, el bloqueo preventivo degradaría progresivamente la oferta visible del comercio.

- **RF-23 Ciclo de vida del pedido:** el sistema debe gobernar el pedido mediante los estados PENDIENTE_PAGO, PAGADO_CONFIRMADO, RETIRADO, EXPIRADO y CANCELADO, admitiendo exclusivamente las transiciones válidas entre ellos.

1.6. Confirmación de la transacción

La integración con la pasarela se implementa en entorno de pruebas. Esta decisión permite desarrollar y validar el flujo transaccional completo (incluida la comunicación asincrónica y la reversión ante la falta de confirmación) sin que el proyecto asuma obligaciones financieras ni fiscales impropias de un trabajo académico.

- **RF-24 Derivación al pago:** el sistema debe generar la preferencia correspondiente al pedido y derivar al comprador al entorno de *checkout* de MercadoPago, en modalidad Sandbox.
- **RF-25 Consolidación:** el sistema debe recibir la notificación asincrónica de la pasarela y, ante la aprobación del cobro, consolidar el pedido en estado PAGADO_CONFIRMADO.
- **RF-26 Idempotencia:** la recepción reiterada de un mismo evento de notificación no debe producir confirmaciones ni descuentos duplicados.
- **RF-27 Emisión del comprobante:** confirmada la operación, el sistema debe emitir el comprobante de retiro (*ticket*) identificado mediante un código único, que consigne el detalle del pedido, el comercio y la franja horaria asignada.

1.7. Cierre de la operación y comunicación

WhatsApp conserva su lugar como canal de contacto entre las partes, pero deja de ser el ámbito donde se negocia la operación. Al momento de intervenir, el pedido ya se encuentra definido, confirmado y agendado.

- **RF-28 Envío del comprobante:** el sistema debe habilitar el envío del comprobante de retiro a través de WhatsApp mediante enlace estructurado, conectando al comprador con el vendedor para la coordinación del encuentro.
- **RF-29 Panel de la jornada:** el vendedor debe poder visualizar los pedidos del día organizados por estado y por franja horaria.
- **RF-30 Verificación del retiro:** el vendedor debe poder verificar el código del comprobante al momento de la entrega y registrar el pedido como RETIRADO, cerrando el ciclo de la operación.

1.8. Accesibilidad operativa

Dado el perfil de usuario descrito en el apartado *Contexto y Actores*, la accesibilidad no constituye una capa cosmética sino una condición de uso de la plataforma.

- **RF-31 Controles de visualización:** la interfaz debe proveer controles nativos para activar el modo oscuro, ajustar el tamaño tipográfico y aplicar una paleta de alto contraste.
- **RF-32 Persistencia de preferencias:** las preferencias de visualización seleccionadas deben conservarse entre sesiones para un mismo dispositivo.



Requisitos No Funcionales

Los requisitos no funcionales establecen los atributos de calidad que el sistema debe satisfacer, formulados de manera tal que su cumplimiento resulte verificable durante las pruebas. Las tecnologías adoptadas y su fundamentación se desarrollan en el apartado *Definición del Stack Tecnológico* y no se reiteran aquí.

- **RNF-01 Accesibilidad:** la interfaz debe alcanzar el nivel AA de las Pautas de Accesibilidad para el Contenido Web (WCAG 2.1), lo que supone una relación de contraste mínima de 4.5:1 en el texto, la ausencia de información transmitida exclusivamente mediante color y el soporte de ampliación tipográfica hasta el 200 % sin pérdida de contenido ni desplazamiento horizontal. Su cumplimiento se verifica mediante auditoría automatizada.
- **RNF-02 Usabilidad:** las operaciones críticas del vendedor (publicar un producto y registrar un retiro) deben resolverse en un máximo de tres interacciones y sin requerir conocimientos técnicos previos.
- **RNF-03 Rendimiento:** las consultas de catálogo con filtrado por proximidad deben responder en menos de dos segundos bajo condiciones normales de operación. Las imágenes deben comprimirse en el momento de su carga.
- **RNF-04 Compatibilidad:** la aplicación debe adoptar un diseño responsivo con enfoque *mobile-first* y resultar operable sobre dispositivos de gama media-baja en condiciones de conectividad limitada, escenario predominante entre los usuarios destinatarios.
- **RNF-05 Seguridad:** las credenciales deben almacenarse mediante función de *hash*; la autenticación debe ser *stateless*, con expiración del token, autorización por rol verificada en cada punto de acceso y transporte cifrado.
- **RNF-06 Aislamiento:** ningún comercio debe acceder a datos de gestión pertenecientes a otro. La condición se verifica mediante pruebas automatizadas de acceso cruzado.
- **RNF-07 Integridad transaccional:** el bloqueo preventivo de stock y de cupo debe impedir la sobreventa ante solicitudes concurrentes sobre una misma franja horaria.
- **RNF-08 Tolerancia a fallos:** la indisponibilidad transitoria de la pasarela no debe dejar pedidos en estado inconsistente; los pedidos sin confirmar deben ser recuperados por el mecanismo de liberación automática.
- **RNF-09 Mantenibilidad:** el sistema debe organizarse en capas con separación de responsabilidades, con tipado estático en el cliente y bajo un repositorio único con control de versiones.
- **RNF-10 Disponibilidad:** los componentes de backend y de persistencia deben operar desplegados en la nube y resultar accesibles públicamente.
- **RNF-11 Trazabilidad:** el sistema debe registrar las transiciones de estado de cada pedido con su correspondiente marca temporal.
- **RNF-12 Privacidad:** la recolección de datos personales debe limitarse a lo estrictamente necesario para la operación, en cumplimiento de la Ley 25.326 de Protección de los Datos Personales.



Fuera del Alcance (Out of Scope)

Las exclusiones que siguen responden a dos criterios. El primero comprende aquellas funcionalidades cuya implementación exigiría operar con fondos reales o asumir obligaciones legales incompatibles con un trabajo académico. El segundo abarca capacidades que agregarían valor al producto pero que no resultan necesarias para demostrar la validez del modelo propuesto, y cuya incorporación comprometería la profundidad con que pueden desarrollarse los módulos centrales.

- **Operación con fondos reales.** La integración con la pasarela se limita al entorno Sandbox, lo que permite validar el flujo transaccional completo sin asumir obligaciones financieras ni fiscales.
- **Devoluciones, reembolsos y resolución de disputas.** Su tratamiento resulta indisociable de la operación con fondos reales y se excluye en consecuencia con el punto anterior.
- **Logística y seguimiento de envíos.** No se desarrollarán módulos de distribución a domicilio ni de seguimiento de paquetería: el modelo se sustenta íntegramente en el retiro presencial en el comercio.
- **Plataforma de mensajería interna.** No se construirá un sistema de chat nativo; la comunicación entre las partes se delega a WhatsApp mediante el envío del comprobante estructurado.
- **Integración con la API oficial de WhatsApp Business.** El envío se resuelve mediante enlace estructurado, sin automatización de mensajería ni verificación comercial ante el proveedor.
- **Facturación electrónica e integración con organismos fiscales.** El sistema no emitirá comprobantes con validez fiscal ni se conectará a los servicios web de recaudación, ya que la plataforma actúa únicamente como nexo transaccional tecnológico y el proyecto se enfoca en resolver la problemática logística y de vitrina del comercio informal.
- **Aplicación móvil nativa.** La cobertura del canal móvil se resuelve mediante una aplicación web responsiva.



Análisis de Viabilidad

La justificación para llevar a cabo este proyecto radica en que la infraestructura propuesta no solo es realizable dentro de los plazos académicos, sino que resuelve una problemática urbana concreta y latente, dotando de formalidad y seguridad a un ecosistema altamente ineficiente.

Viabilidad Técnica

El proyecto es plenamente viable a nivel de ingeniería. Las tecnologías seleccionadas (Spring Boot, React, MongoDB) son maduras, cuentan con soporte extenso a largo plazo y se integran con naturalidad. El mayor desafío técnico —la gestión de concurrencia en el inventario— se resuelve de manera óptima utilizando el ecosistema de Spring. El sistema de Gestor de Retiros se apoya en esto para ejecutar un flujo transaccional seguro:

1. **Bloqueo preventivo:** Al seleccionar una franja horaria (ej. 17:00 a 17:30), Spring Boot descuenta el stock temporalmente y marca el cupo en estado PENDIENTE_PAGO.
2. **Delegación y Webhooks:** El usuario navega al Checkout de MercadoPago. Si la pasarela aprueba el cobro, el webhook dispara la consolidación de la orden (PAGADO_CONFIRMADO) y emite el ticket.
3. **Rollback automatizado:** Si el pago no se efectúa, tareas programadas en el servidor (@Scheduled de Spring Boot) detectan la caducidad a los 15 minutos, liberando el stock y el turno para otro vecino.

Viabilidad Operativa

Desde la perspectiva del negocio y del usuario, la ejecución del proyecto es crítica para erradicar el caos del comercio a través de estados de WhatsApp. MeVe otorga viabilidad operativa a los vendedores barriales al devolverles el control de su tiempo. El Gestor de Retiros elimina la incertidumbre logística: si un comerciante configura una capacidad de atención de 3 personas por hora, el sistema bloquea automáticamente los turnos excedentes. A esto se le suma la pasarela de pagos, que aporta un marco de garantías y reembolsos inexistente en el comercio informal actual, fomentando la confianza ciega entre vecinos y blindando la economía local.

Viabilidad Temporal

El alcance del MVP (Minimum Viable Product) ha sido delimitado estratégicamente para asegurar su entrega exitosa. Al dejar fuera del alcance (Out of Scope) funcionalidades periféricas complejas como el tracking logístico de envío. El desarrollo se concentra pura y exclusivamente en los módulos críticos: la arquitectura Multitenant, el Gestor de Retiros y la extrema accesibilidad de las interfaces. Esto garantiza el cumplimiento del cronograma estipulado para la presentación y validación del Trabajo Final Integrador.



Uso Crítico de Herramientas de IA

La integración de la Inteligencia Artificial durante la fase de concepción de este Trabajo Final se realizó bajo un enfoque estrictamente analítico. El modelo de lenguaje no fue empleado para la delegación acrítica de decisiones, sino como un interlocutor técnico para someter a estrés la propuesta y refinar el pensamiento arquitectónico del equipo.

- **Prompting de Rol y Evaluación Arquitectónica:** Se instruyó a la IA para actuar como un Project Manager y Arquitecto de Software experimentado, con el objetivo de identificar puntos ciegos en el flujo del *O2O Commerce*. Esta interacción permitió estructurar conceptualmente el manejo de estados concurrentes (bloqueo preventivo y rollback automatizado), garantizando la integridad transaccional frente a la pasarela de MercadoPago.
- **Definición del MVP y Poda de Alcance:** Se utilizó la IA para evaluar la viabilidad temporal de múltiples ideas de valor agregado aportadas por el equipo. Mediante este análisis de factibilidad, se descartaron módulos de alto riesgo técnico para un desarrollo de tres meses (como un sistema de subastas en tiempo real y métricas de impacto ambiental), consolidando en su lugar el Gestor Inteligente de Retiros como el núcleo viable del producto.
- **Soberanía en la Toma de Decisiones Técnicas:** La IA sugirió iterativamente el uso de frameworks unificados (como Next.js) para acelerar el desarrollo del MVP. Sin embargo, el equipo ejerció su criterio profesional descartando la sugerencia automatizada y priorizando la viabilidad basada en el conocimiento real de los integrantes, optando por mantener una arquitectura separada con Spring Boot para el backend y React con TypeScript para el frontend, garantizando así un desarrollo robusto y sin dependencias de un único entorno.
- **Análisis Competitivo y Diferenciación:** La IA funcionó como asistente de investigación para contrastar la idea inicial contra el contexto real del mercado (ej. operaciones a través de Facebook Marketplace o estados de WhatsApp). Esto permitió estructurar argumentos concretos de diferenciación, definiendo que el valor agregado real residía en la sustitución de la coordinación conversacional por un sistema de asignación de capacidad logística.



Anexo

- Repositorio de Github: <https://github.com/taroutn/meve>

